



UNIVERSIDAD
DE GRANADA

Este documento está protegido por la Ley de Propiedad
Intelectual (Real Decreto Ley 1/1996 de 12 de abril).
Queda expresamente prohibido su uso o distribución
sin autorización del autor.

BiGS

versión 0.1

Biblioteca Gráfica Simple (basada en Allegro 5)

1. Introducción.....	2
2. Un ejemplo sencillo: ¡Hola, mundo!.....	3
3. Instalación y uso de BiGS.....	3
4. Introducción a los sistemas gráficos.....	5
5. BiGS. Primitivas de dibujo.....	7
6. Control del teclado y del ratón.....	9
7. Documentación de la API.....	13
8. Compilación y enlazado en línea de órdenes.....	20

© Prof. Javier Martínez Baena
Dpto. Ciencias de la Computación e I. A.
Universidad de Granada



Departamento de
Ciencias de la Computación
e Inteligencia Artificial

1. Introducción

Generalmente, las asignaturas de programación se ciñen al estudio de algún lenguaje de programación de alto nivel (C++ en nuestro caso), planteándose como meta el conocimiento de las herramientas que proporciona dicho lenguaje para desarrollar soluciones a problemas del mundo real.

El uso de interfaces para la comunicación con el usuario suele quedar relegado a un segundo plano o a cursos más avanzados. Esto tiene sentido, ya que lo primero es necesario conocer técnicas de programación básicas y familiarizarse con los lenguajes. Además, la gestión de interfaces con el usuario tiene su propia complejidad intrínseca, por lo que su estudio a niveles tempranos distraería al estudiante de los objetivos de un curso de programación básica. Por estos motivos, la comunicación con el usuario suele hacerse mediante la consola y en modo texto, simplificándola al máximo. Obviamente las posibilidades que ofrece la consola son limitadas, por ejemplo no permite diseñar programas con interfaces gráficas. Sin embargo, es mucho más atractivo para el usuario de las aplicaciones y posiblemente para el estudiante hacer programas que visualmente sean parecidos a cualquier aplicación de uso cotidiano que tenga instalada en su ordenador.

Existe una gran variedad de bibliotecas orientadas al desarrollo de aplicaciones gráficas: SDL¹, SFML², Raylib³, etc. Cada una de ellas tiene ventajas e inconvenientes pero podríamos afirmar que ninguna es apta para el contexto de esta asignatura (o, al menos, podemos decir que su uso no sería trivial).

Teniendo en cuenta todo esto, se ha desarrollado una biblioteca gráfica apta para su uso por estudiantes de primer curso que evita cualquier complejidad técnica y que permite centrarse en el aprendizaje de la programación, pero facilitando el desarrollo de aplicaciones gráficas. No se ha implementado desde cero sino que se ha hecho haciendo uso de otra ya existente. Concretamente lo que se ha implementado es una capa de abstracción (*wrapper*) muy sencillo sobre Allegro 5⁴. Algunas características de Allegro 5 son estas:

- Es software libre y multiplataforma (GNU/Linux, Windows, MacOS, Android, iPhone, ...).
- Está desarrollada en C. Es compatible con C++.
- Es un software muy veterano y bastante robusto. Comenzó a desarrollarse a mediados de los '90.
- Y diversas cuestiones técnicas: uso de aceleración por hardware, OpenGL, *shaders*, ...

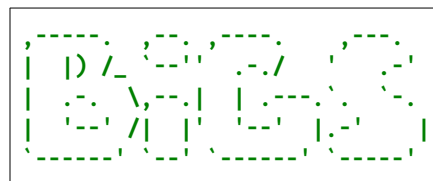
Allegro se autodefine como “*A game programming library*”, es decir, que está pensada para el desarrollo de videojuegos aunque se puede utilizar para crear cualquier otro tipo de aplicación. Puede ver algunos ejemplos de videojuegos desarrollados con este software aquí:

<https://itch.io/games/made-with-allegro>

En la capa que se ha implementado solo se contempla el uso (simplificado) de algunas facetas de Allegro:

- Dibujo de gráficos sencillos (puntos, líneas, círculos, rectángulos y elipses).
- Bitmaps. Carga desde ficheros (png, jpg, etc.) y dibujo en la ventana.
- Eventos. Captura de eventos de teclado, ratón y ventana.
- Obtención del estado del teclado en tiempo real.
- Fuentes. Carga de fuentes desde ficheros (ttf, otf, etc.)
- Audio. Carga desde ficheros (ogg, wav, etc.) y reproducción.
- Temporizador.

A esta nueva capa sobre Allegro 5 la hemos llamado BiGS, acrónimo de **B**iblioteca **G**ráfica **S**imple.



1 <https://www.libsdl.org/>

2 <https://www.sfm1-dev.org/>

3 <https://www.raylib.com/>

4 <https://liballeg.org/>

2. Un ejemplo sencillo: ¡Hola, mundo!

```
#include "bigs.h"
int main(int argc, char *argv[]) {
    BiGS::windowOpen(300,180,"BiGS: ¡Hola, mundo!");
    BiGS::drawRectangle(80,70,220,100,BiGS::CRed);
    BiGS::txtDraw("¡Hola, mundo!",100,80,BiGS::CGreen);
    BiGS::eventWaitKeyboard();
    return 0;
}
```



Los diferentes elementos de este programa son los siguientes:

- Se incluye el fichero de cabecera “bigs.h” con todas las definiciones de la biblioteca. Todo está incluido en un espacio de nombres llamado **BiGS** y por ello se antepone **BiGS::** a todo elemento de esta biblioteca. Podría hacer `using namespace BiGS` para abreviar la escritura del código.
- Antes de poder dibujar nada hemos de abrir una ventana gráfica con `windowOpen()`. Indicamos el título de la ventana y el tamaño en píxeles de la misma.
- Usamos dos primitivas de dibujo: una para hacer un rectángulo rojo y otra para escribir un texto.
- Usamos `eventWaitKeyboard()` para esperar a que el usuario pulse una tecla antes de terminar y cerrar la ventana.
- Para que sea posible enlazar correctamente el código objeto de este `main()` es obligatorio usar el prototipo completo tal y como ve en el ejemplo: incluyendo los parámetros `argc` y `argv`, aunque no se utilicen en nuestro programa. Esta es una restricción impuesta por Allegro.

3. Instalación y uso de BiGS

Este módulo se suministra en un fichero llamado **bigs.tgz**. Para instalarlo y utilizarlo en las prácticas simplemente hemos de descomprimirlo en un subdirectorio de nuestro directorio de trabajo en la asignatura. Por ejemplo, si tenemos ubicadas las prácticas en `~/MP/src` ejecutaremos lo siguiente:

```
Prompt> cd ~/MP/src
Prompt> tar xzvf bigs-<SO>.tgz
```

En la página de la asignatura tiene BiGS compilado para diversos sistemas operativos. Elija el adecuado.

Esto habrá creado dos subdirectorios. Uno de ellos llamado **bigs** con la biblioteca y otro llamado **bigs.test** con un proyecto de prueba que hace uso de la biblioteca. Con más detalle, la estructura es la siguiente:

```

bigs
├── bigs.cmake
├── bin
├── data
├── doc
├── include
├── lib
├── src
└── bigs.test
    ├── CMakeLists.txt
    └── *.cpp
```

en donde:

- **src/** contiene varios ficheros fuente con ejemplos de uso de BiGS.
- **bin/** contiene los ejecutables de los ficheros de **src**. Están compilados en GNU/Linux.
- **data/** contiene algunos ficheros de datos de ejemplo (bitmaps, audios y fuentes).
- **doc/** contiene este documento.
- **lib/** contiene el fichero con la biblioteca compilada. Este será enlazado con las aplicaciones que desarrolle.
- **include/** contiene los ficheros de cabecera necesarios para compilar sus aplicaciones.
- **big.s.cmake** es el fichero que deberá incluir en su **CMakeLists.txt** para que CMake sepa como compilar y enlazar sus aplicaciones usando esta biblioteca.

El subdirectorio **big.s.test** contiene los ejemplos de este documento contruidos como un proyecto independiente. Se proporciona para que vea cómo utilizar BiGS en nuevos proyectos. Hay un ejemplo de fichero **CMakeLists.txt** con el siguiente contenido:

```
cmake_minimum_required(VERSION 3.12)
project(BiGStest VERSION 0.1.0 LANGUAGES C CXX)
set(CMAKE_CXX_STANDARD 17)
include(..big.s/big.s.cmake)           # Incluir fichero de configuración de BiGS
add_executable(hola hola.cpp)
target_link_libraries(hola big.s)      # Añadir dependencia
```

Puede ver que se hace una inclusión del fichero **big.s.cmake** proporcionado por la biblioteca. Con esto ya disponemos de un target para CMake llamado **big.s** para añadir como dependencia a cualquier nuevo módulo que lo necesite.

Importante

Es importante que su programa cumpla dos requisitos para que todo funcione correctamente:

1. En el mismo fichero en donde se va a implementar la función **main()** debe hacerse el siguiente *include*:

```
#include "big.s.h"
```

2. El prototipo de la función **main()** debe ser, exactamente, el siguiente:

```
int main(int argc, char *argv[]);
```

Si no lo hace así, es posible que obtenga un error al ejecutar su programa similar a este:

```
dyld[4919]: symbol not found in flat namespace '__al_mangled_main'
```

3.2. Requisitos adicionales

Como ya se ha indicado, BiGS hace uso de la biblioteca Allegro por lo que es necesario tenerla también instalada en el sistema para poder compilar nuevos proyectos que hagan uso de ella (utilizando sus ficheros de cabecera **.h**) y para poder ejecutarlos (usando las bibliotecas dinámicas). Puede descargar los fuentes y compilarlos o bien descargar los binarios desde su web pero se recomienda utilizar el sistema de instalación de paquetes de su Sistema Operativo puesto que lo normal será que esté disponible en los repositorios habituales.

GNU/Linux

En este sistema la instalación se hará con apt:

```
apt update
apt install liballegro5-dev
```

Los ficheros se instalan en diversos directorios del sistema. Si necesitase consultar los ficheros de cabecera o los de CMake puede hacerlo en estas ubicaciones:

```
/usr/include/allegro5
/usr/lib/cmake/allegro
```

Windows con WSL

La instalación de Allegro es la misma que para un sistema GNU/Linux. Debe tener en cuenta que para poder ejecutar aplicaciones gráficas se necesita WSL2 (no funcionan con WSL1).

Windows con MSYS

La instalación se hará con:

```
pacman -S mingw-w64-ucrt-x86_64-allegro
```

MacOS

La instalación se puede hacer con la herramienta Homebrew:

```
brew install allegro
```

Los ficheros se instalan en `/usr/local/Cellar/allegro`

4. Introducción a los sistemas gráficos

Cuando estamos trabajando en modo gráfico podemos ver la pantalla como un lienzo en el que podemos pintar cualquier cosa (texto, líneas, puntos, etc). Nuestro pincel, en este caso, serán los distintos subprogramas que permiten mostrar algo en la pantalla. Habrá subprogramas que dibujen un único punto, otros que dibujen líneas, otros que dibujen formas más complejas, etc.

Con esta biblioteca nuestro lienzo consistirá en una única ventana. La primitiva gráfica que permite crear la ventana se llama `windowOpen()`.

Por ejemplo, la siguiente llamada:

```
windowOpen(200,100,"Mi ventana");
```

inicializa el sistema gráfico y crea una ventana titulada "Mi ventana" que tiene de ancho 200 puntos y de alto 100 puntos.

4.1. Los puntos (píxeles)

Este lienzo (la ventana) tiene la forma de una matriz bidimensional de manera que cada celda de esa matriz representa un punto de la pantalla (estos puntos se llaman "píxel"). Habitualmente se usa un sistema de coordenadas denominado "izquierdo"; en este sistema el $(0,0)$ se sitúa en la esquina superior izquierda y avanzamos hacia abajo y hacia la derecha (ver figura). El tamaño del lienzo, es decir, el número total de puntos, lo decidiremos nosotros cuando inicializamos el sistema gráfico.



Puesto que en C++ los vectores y matrices comienzan a numerarse en 0, las dimensiones de una ventana de tamaño filas x columnas irán desde 0 hasta filas-1 y columnas-1 respectivamente.

4.2. Los colores

Para hacer un dibujo no basta con indicar únicamente la posición en la que hay que pintar. También hemos de indicar el color de los píxeles que se van a dibujar. El color de cada píxel se forma mediante la mezcla de tres colores básicos: rojo, verde y azul. Este sistema de color (aditivo) se conoce como RGB (Red, Green, Blue) y es el que habitualmente se usa para representar los colores en los monitores. Lo que hemos de hacer para decidir el color de un punto es indicar en qué proporción hemos de mezclar esos tres colores. La medida de cada color básico en esa mezcla nos la da un número que varía en el rango $[0,255]$ (en algunos sistemas puede ser

un número real entre 0.0 y 1.0). Un valor de cero indica que ese color no influye en la mezcla y un valor de 255 dice que el color influye al máximo; valores intermedios indican mayor o menor presencia del color básico en la mezcla. Por ejemplo, si deseamos obtener un pixel de color azul puro hemos de usar la combinación (rojo=0, verde=0, azul=255): con esto estamos diciendo que únicamente usaremos el color básico azul con la máxima intensidad. Si deseamos un azul menos luminoso podemos definirlo, por ejemplo, mediante la tripleta (0,0,128) indicando que sólo usaremos la componente azul pero a la mitad de intensidad. En la figura puedes ver algunos ejemplos de composiciones de color.

Color	Muestra	Rojo	Verde	Azul
Amarillo		255	255	0
Naranja		255	128	0
Magenta		255	0	255
Cyan		0	255	255
Rojo		255	0	0
Verde 1		0	255	0
Verde 2		0	190	0
Verde 3		0	80	0
Blanco		255	255	255
Gris		128	128	128
Negro		0	0	0

Observa que, usando estas tres componentes de color, cada una de ellas en el rango [0,255] podemos representar una gama de $256 \times 256 \times 256 = 16.777.216$ colores. Esta gama es más que suficiente para la visualización de cualquier imagen si tenemos en cuenta que el ojo humano tiene capacidad para distinguir aproximadamente unos 10 millones de tonos diferentes como máximo.

4.3. Los dibujos

La tarea de dibujar en una pantalla consiste, básicamente, en determinar qué píxeles deseamos encender y con qué intensidades de rojo, verde y azul (es decir, con qué color).

Por ejemplo, para pintar una línea existen algoritmos que determinan la secuencia de píxeles que hemos de activar, en esa matriz bidimensional, para que parezca que hay una línea. Si nos fijamos con detalle, veremos que la línea no es continua sino que, efectivamente, es una secuencia de puntos más o menos diferenciados (esto dependerá en parte de la resolución del monitor que estemos usando).

Estos algoritmos de dibujo suelen ser bastante laboriosos a pesar de que la tarea pueda parecer trivial. En la figura puedes observar una ventana en la que se han dibujado (con el tamaño de los píxeles aumentado) dos líneas y dos círculos. La línea roja va desde el punto (3,3) hasta el (8,8). La línea verde va desde el punto (3,12) hasta el (12,8). El círculo pequeño estaría centrado en (12,3) y tendría radio 2. El círculo grande estaría centrado en (22,8) y tendría radio 5.



4.4. El texto

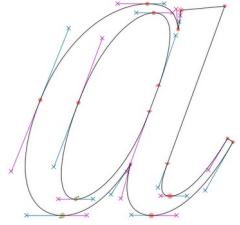
La forma en la que se escriben textos en una ventana gráfica no difiere, en esencia, del dibujo de cualquier otra cosa. Por ejemplo, para que podamos ver una letra 'a' en la pantalla hemos de colorear algunos de los píxeles de forma que en conjunto aparenten formar dicha letra. Lo habitual es que para construir los caracteres se utilicen pequeñas matrices bidimensionales, tal que en cada una se marcan cuáles serían los puntos que se deben encender para cada letra. Por ejemplo, en la figura de la derecha vemos varios ejemplos de letras pintadas punto a punto. Podemos ver la letra 'b' en verde y la letra 'a' con dos tipos distintos de letra (o fuentes de caracteres). Esta es una forma muy simple de definir nuevos conjuntos de caracteres (fuentes): tenemos una matriz para cada letra y al escribir un texto vamos pintando puntos de acuerdo a las matrices que forman cada uno de los caracteres.

El tamaño de estas matrices variará en función del tipo de letra o el tamaño. En el ejemplo anterior hemos usado una matriz de tamaño 8x10. A este tipo de fuente se la conoce como *rasterizada* o fuente de *bitmap*.

4.4.1. Los tipos de letra vectoriales

Un inconveniente de las fuentes de tipo *rasterizado* es que a partir del patrón original de cada letra, no es fácil cambiar el tamaño o el estilo (negrita, itálica, ...). Normalmente, la calidad de una de estas fuentes, tras cambiar su estilo o tamaño, es bastante mala. Para solucionarlo surge una nueva forma de representar las letras.

Las fuentes de tipo vectorial (también conocidas como escalables), definen cada letra mediante una serie de trazos de manera que al conectarlos entre sí dan la sensación de continuidad y nos permiten ver las letras. En la figura de la derecha vemos una letra 'a' con una serie de puntos y marcas superpuestos. Esos puntos (llamados puntos de control) sirven para ir definiendo cada uno de los trozos de curva que componen la letra mediante una función polinómica que, en conjunto, están definiendo por trozos la forma completa (véase splines⁵ o curvas de Bézier⁶). Al tratarse de funciones continuas podemos escalarlas al tamaño que deseemos sin perder calidad. En el caso de las fuentes rasterizadas tenemos que inventarnos píxeles si aumentamos el tamaño y el resultado suele ser malo. Una vez que tenemos los parámetros de los polinomios a la escala adecuada podemos aplicar algún algoritmo de rasterizado que calcule cuales son los píxeles por los que pasa la curva. Algo parecido ocurre si queremos cambiar el estilo de la fuente: negrita, itálica, etc.



Por supuesto, escribir con este tipo de fuentes es computacionalmente más costoso, sin embargo, la calidad es muy superior y en la actualidad es lo habitual. Seguramente le suene el término "*TrueType*": se trata de un estándar desarrollado por Apple para definir fuentes vectoriales y muy posiblemente las fuentes que utiliza en su ordenador están escritas en él. Si ve un fichero con extensión *ttf* ya sabe que almacena la definición de un tipo de fuente *TrueType*. Existen formatos alternativos como *OpenType* (desarrollado por Microsoft) y *FreeType* (*open source*).

5. BiGS. Primitivas de dibujo

La biblioteca BiGS proporciona algunas primitivas para dibujar formas geométricas sencillas tales como puntos, líneas, rectángulos, círculos y elipses. En el ejemplo "¡Hola, Mundo!" hemos visto un ejemplo de uso de la función que dibuja rectángulos.

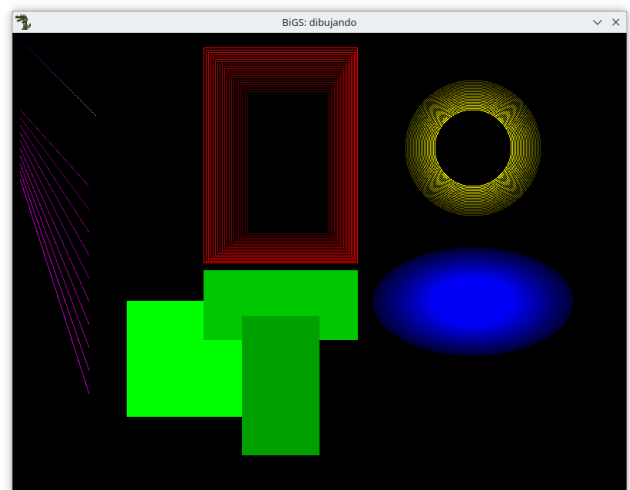
Este otro ejemplo muestra el uso de otras primitivas de dibujo junto con una captura de pantalla del resultado:

```
#include "bigs.h"
using namespace BiGS;
int main(int argc, char *argv[]) {
    windowOpen(800,600,"BiGS: dibujando");

    // Dibujamos puntos en tonos azules
    for (int i=0; i<50; ++i) {
        Color c = colorRGB(i*4,i*4,250);
        drawPoint(10+i*2,10+i*2,c);
    }

    // Dibujamos líneas rectas (tonos violeta)
    for (int i=0; i<10; ++i) {
        Color c = colorRGB(100+i*10,0,100+i*10);
        drawLine(10,100+i*10,100,200+i*30,c);
    }

    // Dibujamos rectángulos (tonos rojos)
    for (int i=0; i<20; ++i) {
        Color c = colorRGB(255-i*10,0,0);
        drawRectangle(250+i*3,20+i*3,450-i*2,300-i*2,c);
    }
}
```



5 <https://es.wikipedia.org/wiki/Spline>

6 https://es.wikipedia.org/wiki/Curva_de_B%C3%A9zier

```

// Dibujamos rectángulos con relleno (tonos verdes)
drawRectangle(150,350,350,500,colorRGB(0,255,0),colorRGB(0,255,0));
drawRectangle(250,310,450,400,colorRGB(0,200,0),colorRGB(0,200,0));
drawRectangle(300,370,400,550,colorRGB(0,160,0),colorRGB(0,160,0));

// Dibujamos círculos sin relleno (tonos amarillos)
for (int i=0; i<20; ++i)
    drawCircle(600,150,50+i*2,colorRGB(255-i*10,255-i*10,0));

// Dibujamos elipses con relleno (tonos azules)
for (int i=20; i>0; --i)
    drawEllipse(600,350,50+i*4,30+i*2,colorRGB(0,0,255-i*10),colorRGB(0,0,255-i*10));

// Esperamos a que se pulse una tecla
eventWaitKeyboard();
return 0;
}

```

Todas estas funciones hacen los dibujos en la ventana que debemos haber abierto previamente. En el caso de que realicemos dibujos que se salen de las coordenadas físicas de esa ventana, el resultado es indeterminado. En la sección 7 dispone de un listado completo de todas las funciones disponibles en BiGS.

La función `openWindow()` puede llevar algunos parámetros adicionales. El prototipo completo es este:

```

void windowOpen(int w, int h, const char *tit="BiGS app", bool antialias=false,
                int mode=DRAWMODE_INSTANT);

```

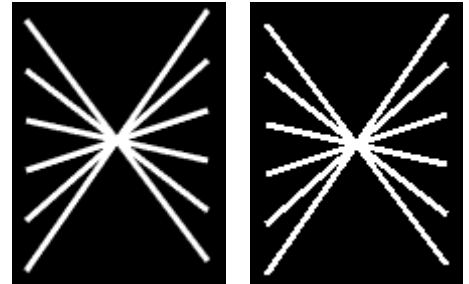
Puede ver que los últimos parámetros llevan valores por defecto por lo que no es obligatorio utilizarlos.

El parámetro `antialias` indica si los algoritmos de dibujo deben o no aplicar una técnica para hacer las formas más suaves (menos pixeladas). Podría ocurrir que su hardware no soporte esta técnica aunque lo más probable es que sí. Pruebe el siguiente ejemplo cambiando ese valor a `true` o `false`.

```

#include "bigs.h"
using namespace BiGS;
int main(int argc, char *argv[]) {
    windowOpen(200,200,"BiGS: antialiasing",true);
    for (int i=0; i<6; ++i)
        drawLine(10,40+i*25,100,160-i*25,CWhite,3);
    eventWaitKeyboard();
    return 0;
}

```

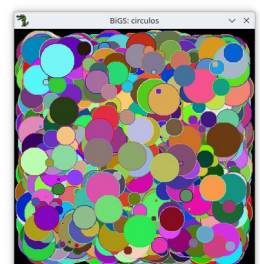


Normalmente, la ejecución de las funciones de dibujo es bastante lenta en comparación con el resto de operaciones que podemos hacer con la CPU. Por defecto, si ejecutamos cualquiera de ellas vemos el efecto de manera inmediata en la ventana, es decir, justo tras ejecutar una llamada a la función `drawPoint()`, vemos un punto dibujado en la pantalla. Es lo esperado. Si dibujamos muchos puntos, vemos que el proceso puede ser lento (y se verán los puntos aparecer poco a poco). Esto se debe a que los accesos al hardware de vídeo son relativamente lentos. El siguiente código muestra un ejemplo en el que se dibuja un número elevado de círculos con colores y posiciones aleatorias:

```

#include "bigs.h"
using namespace BiGS;
int main(int argc, char *argv[]) {
    const int W = 400, H=400;
    windowOpen(W,H,"BiGS: circulos");
    const int RMAX = 30;
    for (int i=0; i<10000; ++i)
        drawCircle(RMAX+rand()%(W-2*RMAX),RMAX+rand()%(H-2*RMAX),rand()%RMAX,

```



```

        colorRGB(rand()%255,rand()%255,rand()%255),
        colorRGB(rand()%255,rand()%255,rand()%255));
    eventWaitKeyboard();
    return 0;
}

```

Verá los círculos dibujarse poco a poco y puede tardar varios segundos en completarse el dibujo completo. Imagine que está desarrollando un videojuego en el que se necesita tener la sensación de tiempo real en el movimiento de los personajes que aparecen en pantalla. Eso quiere decir que entre *frame* y *frame* no puede pasar demasiado tiempo. Es frecuente hablar de tasas de refresco 60 FPS (*frames* por segundo), es decir, que dibujamos 60 fotogramas por segundo. Y para cada fotograma hay que dibujar todo lo que tiene que verse en la pantalla: todas las formas, personajes, textos, etc.

BiGS ofrece diversas formas de dibujar. La primera de ellas es la que hemos visto: cada vez que se llama a una función de dibujo vemos el efecto al momento en la ventana. El último parámetro de la función `windowOpen()`, `mode`, sirve para cambiar el modo de dibujo. En los ficheros de cabecera de BiGS se definen los tres posibles que puede tomar:

```

#define DRAWMODE_INSTANT 1
#define DRAWMODE_DELAYED 2
#define DRAWMODE_BACKBUFFER 3

```

- **DRAWMODE_INSTANT.** Actualiza la ventana tras cada operación de dibujo. Utiliza el modelo implementado en Allegro de *buffer* simple.
- **DRAWMODE_DELAYED.** Con este modo las operaciones de dibujo no tienen efecto de forma inmediata. Cuando se llama a la función `refresh()` es cuando se actualiza la ventana con todo lo que se haya dibujado hasta el momento. BiGS mantiene un *bitmap* interno para realizar los dibujos y cuando se llama a `refresh()` es cuando se vuelca este en la ventana. Se usa internamente el modelo de doble *buffer* de Allegro⁷ ⁸ aunque usando un *buffer* previo de dibujo (podríamos hablar de triple *buffer*).
- **DRAWMODE_BACKBUFFER.** Es parecido al modo `DELAYED` ya que hasta que no se llama a `refresh()` no se actualiza la ventana. La diferencia estriba en que con el modo `DELAYED`, cada vez que se llama a `refresh()` se actualiza la ventana con todo lo dibujado desde el comienzo del programa mientras que con el modo `BACKBUFFER` al llamar a `refresh()` se actualiza la ventana solo con aquello que se dibujase desde la anterior llamada a `refresh()` y no se garantiza que se mantenga lo anterior. Se utiliza el doble *buffer* de Allegro sin *bitmap* intermedio. Este modo requiere borrar la ventana cada vez que dibujamos un *frame* ya que no se garantiza que se mantenga lo dibujado en el *frame* anterior. Este modo no debería utilizarlo normalmente.

Salvo para hacer dibujos sencillos, se recomienda usar el modo `DRAWMODE_DELAYED` por cuestiones de eficiencia. Este modo implica llamar a la función `refresh()` cada vez que se necesite refrescar la ventana y que normalmente será cada vez que se haya completado el dibujo de un *frame* completo. Pruebe a cambiar el modo con la llamada:

```

windowOpen(W,H,"BiGS: círculos",true,DRAWMODE_DELAYED);

```

Si hace este cambio verá la ventana negra: no se dibuja nada. Para resolverlo, añada la siguiente línea justo antes de `eventWaitKeyboard()`:

```

refresh();

```

6. Control del teclado y del ratón

Los eventos son un recurso ampliamente utilizado en la programación de aplicaciones gráficas. Permiten que el programa trabaje de forma asíncrona entre distintas partes del código. Por ejemplo, mientras estamos haciendo algún cálculo en nuestro código, pueden estar ocurriendo sucesos que no queremos perder como pulsaciones de teclas, movimientos del ratón, acciones

7 <https://medium.com/@alex.omegapy/double-buffering-ensuring-smooth-animation-and-responsiveness-in-graphics-8ff310ed386e>

8 https://es.wikipedia.org/wiki/Buffer_m%C3%BAltiple

sobre la ventana (minimizar, cerrar, etc.) o incluso peticiones a través de la red. A esos sucesos se los conoce como eventos y la mayor parte de ellos los capta el sistema operativo pero es nuestra aplicación la responsable de procesarlos y responder en consecuencia. Como muchos de ellos se van a producir mientras estamos con otras tareas, Allegro se encarga de recogerlos y almacenarlos en lo que se conoce como cola(s) de eventos a la espera de nuestra atención.

El teclado y el ratón se van a gestionar mediante eventos. El siguiente programa consiste en un bucle que itera hasta recibir el evento de cerrar ventana consultando a la función `closeRequested()` (pulsando sobre la X de cierre de la misma):

```
BiGS::windowOpen(500,400,"BiGS: Eventos");
BiGS::logEvents();
BiGS::txtDraw("Pulsa una tecla",10,10,BiGS::CGreen);
int nlin = 0; // Línea por la que vamos escribiendo
int sizelin = BiGS::txtLineHeight()+1;
int height = BiGS::windowHeight();
// Bucle típico: mientras no se pulse cerrar ventana
while (!BiGS::closeRequested()) {
    BiGS::Event e;
    if (BiGS::eventNext(e)) {
        int nlineas = cuentalineas(BiGS::to_string(e).c_str())+1;
        if (height-nlin*sizelin < 0 ) {
            BiGS::clear();
            nlin = 0;
        }
        BiGS::txtDrawMultiline(BiGS::to_string(e).c_str(),10,nlin*sizelin,480,BiGS::CGreen);
        nlin += nlineas;
    }
}
```

En cada iteración se consulta la cola de eventos con `eventNext()`. Si esta función devuelve `true` significa que hay un evento (emitido por el teclado o por el ratón) pendiente y lo devuelve a través del argumento. Si no hay eventos devuelve `false`. En caso de que haya algún evento muestra información sobre él usando la función `to_string()` para obtener una cadena con la citada información. Además, al comienzo del código se ha ejecutado `logEvents()` que sirve para mostrar en consola los eventos que se van recibiendo.

Existe otra función parecida a `eventNext()` que es `eventWait()`. Esta también comprueba si hay o no eventos pendientes pero es bloqueante, es decir, si al llamarla hay algún evento lo devuelve pero si no hay se espera hasta que lo haya.

También podemos precisar si esperamos eventos de ratón o de teclado con las funciones bloqueantes `eventWaitKeyboard()` y `eventWaitMouse()`. Ambas devuelven el siguiente evento de ratón/teclado y en caso de que no haya ninguno bloquean el programa hasta que lo haya.

Por último hay una función `eventFlush()` que lo que hace es borrar todos los eventos que haya pendientes de procesar.

6.1. El tipo Event

Cuando alguna de estas funciones devuelve un evento nuestro programa debería comprobar cuál es y actuar en consecuencia. En BiGS este tipo es un `typedef` del tipo `ALLEGRO_EVENT` por lo que se puede consultar la documentación de Allegro para ver lo que contiene.

Una variable de tipo `Event` tiene un campo `type` con el tipo de evento que almacena. Dependiendo del que se trate puede consultar unos campos u otros. En la siguiente tabla vemos la información que contiene un evento dependiendo del tipo de evento:

KEY_DOWN KEY_UP	KEY_CHAR	MOUSE_MOVE	MOUSE_BUTTON_DOWN MOUSE_BUTTON_UP	MOUSE_ENTER MOUSE_LEAVE
e.keyboard.keycode	e.keyboard.keycode e.keyboard.unichar e.keyboard.repeat	e.mouse.x e.mouse.y e.mouse.z	e.mouse.x e.mouse.y e.mouse.z	v.mouse.x e.mouse.y e.mouse.z

	e.keyboard.modifiers	e.mouse.w e.mouse.dx e.mouse.dy e.mouse.dz e.mouse.dw e.mouse.pressure	e.mouse.w e.mouse.button e.mouse.pressure	e.mouse.w
--	----------------------	---	---	-----------

El significado de los diferentes eventos es el siguiente:

- KEY_DOWN. Se ha pulsado una tecla.
- KEY_UP. Se ha soltado una tecla.
- KEY_CHAR. Se ha recibido un carácter (como consecuencia de pulsar una tecla).
- MOUSE_MOVE. Se ha movido el ratón.
- MOUSE_BUTTON_DOWN. Se ha pulsado un botón del ratón.
- MOUSE_BUTTON_UP. Se ha soltado un botón del ratón.
- MOUSE_ENTER. El ratón ha entrado en la ventana.
- MOUSE_LEAVE. El ratón ha salido de la ventana.

El siguiente código muestra un ejemplo:

```
while (!BiGS::closeRequested()) {
    BiGS::Event e;
    if (BiGS::eventNext(e)) {
        if (e.type==BiGS::KEY_DOWN) {
            cout << " Evento: KEY_DOWN" << endl;
            cout << " Has pulsado la tecla " << e.keyboard.keycode;
            cout << " (" << BiGS::eventKeyName(e.keyboard.keycode) << ")" << endl;
            if (e.keyboard.keycode == ALLEGRO_KEY_A)
                cout << " ... acabo de comprobar que has pulsado la tecla A" << endl;
        } else if (e.type==BiGS::KEY_CHAR) {
            cout << " Evento: KEY_CHAR" << endl;
            cout << " Has pulsado la tecla " << e.keyboard.keycode;
            cout << " (" << BiGS::eventKeyName(e.keyboard.keycode) << ")" << endl;
            cout << " unichar = " << e.keyboard.unichar << endl;
            if (e.keyboard.modifiers & ALLEGRO_KEYMOD_SHIFT) cout << "SHIFT" << endl;
            if (e.keyboard.modifiers & ALLEGRO_KEYMOD_CTRL) cout << "CTRL" << endl;
            if (e.keyboard.modifiers & ALLEGRO_KEYMOD_ALT) cout << "ALT" << endl;
        } else if (e.type==BiGS::MOUSE_MOVE) {
            cout << "Evento: MOUSE_MOVE" << endl;
            cout << " Has movido el ratón. Coordenadas " << e.mouse.x << ", " << e.mouse.y << endl;
        }
        cout << endl;
    }
}
```

Vemos como tiene sentido acceder a unos u otros campos dependiendo del evento. En el caso del teclado el campo `keycode` tiene un valor con un código que representa la tecla que se ha pulsado. Los posibles valores los establece Allegro 5 y pueden consultarse en la sección 7.6.3. Se ha usado uno de ellos (`ALLEGRO_KEY_A`) a modo de ejemplo para comprobar que se ha pulsado la tecla A.

Hay otro evento que puede confundirse con `KEY_DOWN` que es `KEY_CHAR`. Este último se produce cuando la tecla que se ha pulsado se corresponde con un carácter o, dicho de otra forma, cuando lo que se ha pulsado es una tecla que puede representarse con UNICODE (o ASCII; etc.). Para ver la diferencia pruebe a pulsar la tecla A. Verá que se reciben 2 eventos:

- Evento `KEY_DOWN` que indica que se ha pulsado la tecla A. Este tecla tiene un `keycode` con valor 1.
- Evento `KEY_CHAR` que indica que se ha recibido el carácter 'a'. En este caso se escribe también el código UNICODE de esa letra que es 97 y que se almacena en el campo `unichar`.

Si ahora prueba a pulsar la tecla A pero combinada con la tecla de mayúsculas verá estos tres

eventos:

- Evento `KEY_DOWN` que indica que se ha pulsado la tecla mayúsculas (LSHIFT) cuyo `keycode` es 215.
- Evento `KEY_DOWN` que indica que también se ha pulsado la tecla A cuyo `keycode` es 1.
- Evento `KEY_CHAR` que indica que se ha recibido el carácter 'A' cuyo `keycode` sigue siendo 1 (es la misma tecla que para escribir 'a') y que tiene como código UNICODE el 65.

Si ahora pulsase la tecla mayúsculas y la soltase solo recibiría un evento `KEY_DOWN` pero ninguno de tipo `KEY_CHAR` puesto que esa tecla por sí sola no escribe ningún carácter.

Puede ver como una única acción puede provocar múltiples eventos. En los ejemplos anteriores, aunque no se han comprobado, también se han generado eventos de tipo `KEY_UP` que ocurren cuando soltamos una tecla.

Para los eventos `KEY_CHAR` dispone de dos campos adicionales que pueden ser de interés.

- `e.keyboard.repeat` contiene un valor `true` o `false` que indica si el evento se ha generado porque la tecla se mantiene pulsada. Cuando escribe en un programa cualquiera puede comprobar que si mantiene una tecla pulsada se produce una repetición de la misma y es en esos casos cuando se activa este *flag*.
- `e.keyboard.modifiers` contiene un valor que sirve para saber si una tecla se ha pulsado en combinación con otras (mayúsculas, control, alt, etc.). Este valor se debe interpretar como una secuencia de bits, cada uno de los cuales mantiene el estado de una de estas teclas especiales (o modificadoras). Para consultar los bits individuales se utilizan las operaciones a nivel de bit (`&`, `|`) que no han de confundirse con las operaciones de conjunción y disyunción lógica (`&&`, `||`). Puede consultar la lista de constantes disponibles en 7.6.4.

Para los eventos de ratón puede consultar los siguientes campos:

- `e.mouse.x`, `e.mouse.y`: almacenan la posición del ratón.
- `e.mouse.z`, `e.mouse.w`: almacenan la posición de la rueda del ratón.
- `e.mouse.dx`, `e.mouse.dy`, `e.mouse.dz`, `e.mouse.dw`: almacenan el diferencial de la posición respecto del anterior evento de movimiento del ratón.
- `e.mouse.button` es un código con el botón del ratón que se ha pulsado. Los posibles valores son estos: `ALLEGRO_MOUSE_BUTTON_LEFT`, `ALLEGRO_MOUSE_BUTTON_RIGHT`, `ALLEGRO_MOUSE_BUTTON_MIDDLE`. Sin embargo, si su ratón tiene más botones podrá obtener más códigos numéricos.
- `e.mouse.pressure` es un valor real entre 0 y 1 que indica la presión con la que ha pulsado un botón del ratón.

6.2. Estado del teclado en tiempo real

Hay ocasiones en las que los eventos que gestionan el teclado no son adecuados para las necesidades de un programa. Imagine que debe programar un videojuego y hacer que un personaje responda a la pulsación de las teclas del cursor. Cuando pulsa una tecla para mover el personaje se genera un primer evento `KEY_CHAR` (o `KEY_DOWN`) y a los pocos milisegundos (o incluso segundos) se generan nuevos eventos `KEY_CHAR` debido a la repetición de teclas propia de cualquier teclado. Si nos basamos en esos eventos para mover al personaje perderemos la partida con total seguridad ya que son demasiado lentos. Nosotros queremos que el personaje se mueva mientras se mantiene pulsada una tecla (y no al ritmo que se generan los eventos de pulsación o repetición).

Para resolver este problema disponemos de una función `isKeyPressed(k)` que devuelve `true` si en el momento de llamarla se encuentra pulsada la tecla cuyo `keycode` es `k` y `false` en caso contrario, independientemente del sistema de gestión de eventos.

El siguiente ejemplo ilustra esto:

```
while (!BiGS::closeRequested()) {
    BiGS::Event e;
    if (BiGS::eventNext(e) &&
```

```

        e.type==BiGS::KEY_CHAR &&
        e.keyboard.keycode == ALLEGRO_KEY_A)
        cout << " ... se ha generado un evento de pulsación de la tecla A" << endl;
    if (BiGS::isKeyPressed(ALLEGRO_KEY_A))
        cout << "Atención: la tecla A está pulsada" << endl;
    BiGS::timeDelay(5);
}

```

Se ha incluido una llamada a la función `timeDelay()` para poder apreciar el efecto de las pulsaciones de teclas. De no hacerlo no daría tiempo a ver los mensajes derivados de los eventos. En cada iteración del bucle el programa hace dos cosas:

- Comprobar el estado de la tecla A con `isKeyPressed()` y escribir un mensaje si está pulsada.
- Comprobar si ha llegado un evento `KEY_CHAR` con la tecla A y escribir un mensaje si así es.

Puede comprobar que la generación de eventos es relativamente lenta en comparación con la consulta del estado del teclado.

7. Documentación de la API

7.1. Gestión de la interfaz gráfica

```
void windowOpen(int w, int h, const char *tit="BiGS app", bool antialias=false,
               int mode=DRAWMODE_INSTANT);
```

Abre una ventana. Es obligatorio llamar a esta función antes que a ninguna otra. Solo se permite tener una ventana por aplicación.

`w, h` : Tamaño en píxeles de la ventana.

`tit` : Título de la ventana.

`antialias`: usar o no técnicas de *antialiasing* para todos los dibujos.

`mode`: modo de dibujo. Los posibles valores son: `DRAWMODE_INSTANT`, `DRAWMODE_DELAYED`, `DRAWMODE_BACKBUFFER`

```
int windowWidth();
```

Devuelve el ancho en píxeles de la ventana.

```
int windowHeight();
```

Devuelve el alto en píxeles de la ventana.

```
void refresh();
```

Actualiza la ventana en función de lo establecido por el modo de apertura de la misma. Si el modo es `DRAWMODE_INSTANT` esta operación no tiene efecto. Si el modo es `DRAWMODE_DELAYED` refresca la ventana con todo lo que se haya modificado desde la última llamada a `refresh()`. Si el modo es `DRAWMODE_BACKBUFFER` refresca la ventana con todo lo que haya cambiado desde la última llamada pero no garantiza que se mantenga lo que había dibujado antes.

7.2. Colores

El color es un elemento fundamental en BiGS, todas las primitivas de dibujo lo utilizan. Se ha definido un tipo `Color`, algunas constantes y varias funciones para crear colores.

Los colores predefinidos son estos: `CBlack`, `CWhite`, `CRed`, `CGreen`, `CBlue`, `CCyan`, `CMagenta`, `CYellow`. Además, existe un color `CTransparent` para representar la ausencia de color.

Las siguientes funciones sirven para crear un color en base a los componentes RGB. En unas versiones cada uno es un valor entero entre 0 y 255 y en otras un valor real entre 0 y 1. Además, una de las funciones admite un cuarto valor que representa el nivel de transparencia (*alfa*).

```
Color colorRGB(unsigned char r, unsigned char g, unsigned char b);
```

Crea un color a partir de la tripleta R,G,B. Son tres valores enteros en el intervalo [0,255].

```
Color colorRGBf(float r, float g, float b);
```

Crea un color a partir de la tripleta R,G,B. Son tres valores reales en el intervalo [0,1].

```
Color colorRGBA(unsigned char r, unsigned char g, unsigned char b, unsigned char a);
```

Crea un color a partir de la tripleta R,G,B. Son tres valores enteros en el intervalo [0,255]. Además, incluye un cuarto valor para indicar el nivel de transparencia.

```
Color colorRandom(bool transp=false);
```

Crea un color aleatorio. **transp** indica si se incluye también un nivel de transparencia aleatorio (**true**) o se crea el color sin transparencia (**false**).

7.3. Primitivas de dibujo

```
void clear(Color c);
```

Borra la ventana y la rellena con el color c.

```
void drawPoint(float x, float y, const Color &c);
```

Dibuja un píxel en las coordenadas (x,y) del color c.

```
void drawLine(float x1, float y1, float x2, float y2, const Color &c, float g=1, float ang=0.0);
```

Dibuja una línea de color c entre los puntos (x1,y1) y (x2,y2).

g es el grosor de la línea en píxeles.

ang es la rotación que le aplicará a la línea en radianes. El centro de rotación es el centro de la línea.

```
void drawRectangle(float x1, float y1, float x2, float y2, const Color &c,  
                  const Color &cr=CTransparent, float g=1, float ang=0.0);
```

Dibuja un rectángulo definido entre los puntos (x1,y1) y (x2,y2).

c es el color del borde del rectángulo.

cr es el color de relleno del rectángulo. Si no se desea relleno se puede usar el color CTransparent.

g es el grosor del borde en píxeles.

ang es el ángulo de rotación en radianes. El centro de rotación es el centro del rectángulo.

```
void drawCircle(float x, float y, float r, const Color &c, const Color &cr=CTransparent,  
               float g=1);
```

Dibuja un círculo centrado en (x,y) y de radio r.

c es el color del borde del círculo.

cr es el color de relleno del círculo. Si no se desea relleno se puede usar el color CTransparent.

g es el grosor del borde en píxeles.

```
void drawEllipse(float x, float y, float rx, float ry, const Color &c,  
                const Color &cr=CTransparent, float g=1, float ang=0.0);
```

Dibuja una elipse centrada en (x,y) y de radios rx y ry.

c es el color del borde de la elipse.

cr es el color de relleno de la elipse. Si no se desea rellenarla se puede usar el color CTransparent.

g es el grosor del borde en píxeles.

ang es el ángulo de rotación en radianes. El centro de rotación es el centro de la elipse.

```
void drawTriangle(float x1, float y1, float x2, float y2, float x3, float y3, const Color &c,
                 const Color &cr=CTransparent, float g=1, float ang=0.0);
```

Dibuja un triángulo definido por los puntos (x1,y1), (x2,y2) y (x3,y3).

c es el color del borde del triángulo.

cr es el color de relleno del triángulo. Si no se desea rellenarlo se puede usar el color CTransparent.

g es el grosor del borde en píxeles.

ang es el ángulo de rotación en radianes. El centro de rotación es el baricentro del triángulo.

```
void drawTriangleEqui(float x, float y, float lado, const Color &c, const Color &cr=CTransparent,
                    float g=1, float ang=0.0);
```

Dibuja un triángulo equilátero definido por un punto (x,y) y un lado lado. Por defecto el triángulo apunta hacia abajo.

c es el color del borde del triángulo.

cr es el color de relleno del triángulo. Si no se desea rellenarlo se puede usar el color CTransparent.

g es el grosor del borde en píxeles.

ang es el ángulo de rotación en radianes. El centro de rotación es el baricentro del triángulo.

```
void drawArrow(float x1, float y1, float x2, float y2, float tbase, float tpunta, const Color &c,
               float g=1, float ang=0.0);
```

Dibuja una flecha desde el punto (x1,y1) hasta el punto (x2,y2). En el punto de inicio se puede dibujar un círculo de tamaño tbase y la cabeza de la flecha tendrá tamaño tpunta.

c es el color de la flecha.

g es el grosor de la línea en píxeles.

ang es el ángulo de rotación en radianes. El centro de rotación es el centro de la línea.

7.4. Bitmaps

Un *bitmap* es una imagen. Allegro dispone de una gran funcionalidad para trabajar con imágenes pero en BiGS únicamente se da la opción de cargar imágenes desde ficheros de diversos formatos (jpg, png, etc.) y mostrarlas en la ventana. Para ello se ha implementado una clase **Image** que oculta el detalle de la representación interna de los *bitmaps* y que se puede utilizar a través de las siguientes funciones:

```
Image loadImage(const char *fich);
```

Carga en memoria una imagen desde el fichero con nombre fich. Si hubiese algún problema en la carga se muestra un mensaje de error y se almacena una imagen vacía en el resultado.

```
float imageWidth(const Image &i);
```

Devuelve el ancho en píxeles de la imagen i.

```
float imageHeight(const Image &i);
```

Devuelve el alto en píxeles de la imagen i.

```
Color imagePixel(const Image &i, int x, int y);
```

Devuelve el color del píxel de la posición (x,y) de la imagen i.

```
bool imageEmpty(const Image &i);
```

Devuelve true si i almacena la imagen vacía y false en caso contrario.

```
void imageDraw(const Image &i, float x, float y, float escx=1.0, float escy=1.0, float ang=0.0);
```

Dibuja la imagen i en las coordenadas (x,y) de la ventana.

escx es un factor de escalado en el eje horizontal.

escy es un factor de escalado en el eje vertical.

ang es el ángulo de rotación. El centro de rotación es el centro de la imagen.

```
void imageDrawR(const Image &i, float x1, float y1, float x2, float y2, float ang=0.0);
```

Dibuja la imagen i en el rectángulo definido por (x1,y1)-(x2,y2) de la ventana haciendo el escalado necesario para que la imagen encaje en él.

ang es el ángulo de rotación. El centro de rotación es el centro de la imagen.

7.5. Primitivas de texto

```
void txtDraw(const char *txt, float x, float y, Color c, Font f=FONT_DEFAULT,  
             int aline=ALIGN_LEFT, int ancho=0, float ang=0);
```

Escribe un texto txt en las coordenadas (x,y) de la ventana de color c.

f es la fuente con la que se escribe. Si se pasa el valor FONT_DEFAULT usa la fuente por defecto.

aline permite alinear el texto según uno de estos valores: ALIGN_LEFT, ALIGN_RIGHT, ALIGN_CENTRE, ALIGN_JUSTIFIED.

Si ancho es mayor que cero entonces la justificación es relativa a la caja entre las coordenadas x y x+ancho.

ang es el ángulo de rotación. El centro de rotación es el centro del texto.

```
void txtDrawMultiline(const char *txt, float x, float y, float maxw, Color c, Font f=FONT_DEFAULT,  
                     int aline=ALIGN_LEFT, float lineh=0);
```

Escribe un texto txt en las coordenadas (x,y) de la ventana de color c. Si el texto es muy extenso puede utilizar varias líneas.

f es la fuente con la que se escribe. Si se pasa el valor FONT_DEFAULT usa la fuente por defecto.

maxw es el ancho de la caja en la que debe ajustar el texto.

aline permite alinear el texto según uno de estos valores: ALIGN_LEFT, ALIGN_RIGHT, ALIGN_CENTRE.

lineh es el alto de cada línea en píxeles.

ang es el ángulo de rotación. El centro de rotación es el centro del texto.

```
int txtLineHeight(Font f=FONT_DEFAULT);
```

Devuelve el alto de la fuente f.

```
int txtFontAscent(Font f=FONT_DEFAULT);
```

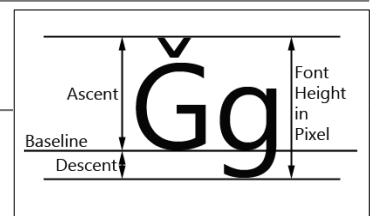
Devuelve el ascendente de la fuente f.

```
int txtFontDescent(Font f=FONT_DEFAULT);
```

Devuelve el descendente de la fuente f.

```
int txtTextWidth(const char *c, Font f=FONT_DEFAULT);
```

Dada una cadena c devuelve el ancho total en píxeles que va a ocupar usando la fuente f.



```
void txtTextDimensions(const char *c, int &dx, int &dy, int &w, int &h, Font f=FONT_DEFAULT);
```

Dada una cadena *c* devuelve el tamaño de la caja que se necesita para mostrarla usando una fuente *f*. En *dx,dy* devuelve el *offset* de la esquina superior izquierda y en *w,h* devuelve el tamaño de dicha caja.

```
Font txtLoadFont(const char *f, int tam=12);
```

Carga una nueva fuente desde el fichero *f*. Una vez cargada no se le puede cambiar el tamaño por lo que cuando se carga se renderiza con el tamaño indicado por *tam*.

7.6. Eventos

Los eventos son un recurso ampliamente utilizado en la programación de aplicaciones gráficas. Permiten que el programa trabaje de forma asíncrona entre distintas partes del código. Por ejemplo, mientras estamos haciendo algún cálculo en nuestro código, pueden estar ocurriendo sucesos que no queremos perder como pulsaciones de teclas, movimientos del ratón, acciones sobre la ventana (minimizar, cerrar, etc.) o incluso peticiones a través de la red. A esos sucesos se los conoce como eventos y la mayor parte de ellos los capta el sistema operativo pero es nuestra aplicación la responsable de procesarlos y responder en consecuencia. Como muchos de ellos se van a producir mientras estamos con otras tareas, Allegro se encarga de recogerlos y almacenarlos en lo que se conoce como cola(s) de eventos a la espera de nuestra atención.

En BiGS solo vamos a disponer de algunos de los eventos manejados por Allegro. Concretamente aquellos relacionados con el teclado, con el ratón y alguno relacionado con la ventana en la que estamos dibujando.

Un evento muy importante en cualquier aplicación es el que se produce al pulsar el aspa para cerrar una ventana. Cuando hacemos eso se genera un evento que en Allegro se conoce como `ALLEGRO_EVENT_DISPLAY_CLOSE`. Pero no se cierra la ventana hasta que nuestro programa lo decida por lo que tenemos que mirar continuamente la cola de eventos y comprobar si se ha pulsado o no el botón. Para evitar eso y simplificar un poco nuestro programa, en BiGS tenemos dos modos de funcionamiento que controlamos con estas dos funciones:

- `closeOnCloseOn()`. Al ejecutarla ponemos a BiGS en un modo en el que, si el usuario pulsa el cierre de la ventana se va a finalizar la ejecución sin que tengamos que hacer nada en nuestra función `main()`.
- `closeOnCloseOff()`. Al ejecutarla ponemos a BiGS en un modo en el que, si el usuario pulsa el cierre de la ventana, la ejecución continua normalmente hasta que nosotros de forma expresa decidamos finalizar. Para saber desde nuestro programa que se ha pulsado ese botón tenemos la función `closeRequested()` que devuelve `true` si así ha sido.

En BiGS hemos renombrado los eventos que vamos a utilizar por simplificar un poco los nombres. Son estos:

```
enum {
    KEY_DOWN = ALLEGRO_EVENT_KEY_DOWN,           // Se pulsa una tecla
    KEY_CHAR = ALLEGRO_EVENT_KEY_CHAR,           // Se recibe un carácter
    KEY_UP = ALLEGRO_EVENT_KEY_UP,               // Se suelta una tecla
    MOUSE_MOVE = ALLEGRO_EVENT_MOUSE_AXES,       // Se ha movido el ratón
    MOUSE_BUTTON_DOWN = ALLEGRO_EVENT_MOUSE_BUTTON_DOWN, // Se pulsa un botón del ratón
    MOUSE_BUTTON_UP = ALLEGRO_EVENT_MOUSE_BUTTON_UP, // Se suelta un botón del ratón
    MOUSE_ENTER = ALLEGRO_EVENT_MOUSE_ENTER_DISPLAY, // El ratón entra en la ventana
    MOUSE_LEAVE = ALLEGRO_EVENT_MOUSE_LEAVE_DISPLAY, // El ratón sale de la ventana
    DISPLAY_EXPOSE = ALLEGRO_EVENT_DISPLAY_EXPOSE, // La ventana se hace visible
    DISPLAY_CLOSE = ALLEGRO_EVENT_DISPLAY_CLOSE, // Se ha pulsado cierre de ventana
    DISPLAY_FOCUS_IN = ALLEGRO_EVENT_DISPLAY_SWITCH_IN, // La ventana gana el foco
    DISPLAY_FOCUS_OUT = ALLEGRO_EVENT_DISPLAY_SWITCH_OUT, // La ventana pierde el foco
    UNKNOWN_EVENT = 1000
```

```
};
```

Estas son algunas funciones para controlar el evento de cierre de la ventana y otras de uso común:

```
void closeOnCloseOn();
```

Indica a BiGS que si recibe el evento de cerrar ventana finalice la ejecución del programa sin necesidad de que controlemos este evento desde nuestra aplicación.

```
void closeOnCloseOff();
```

Indica a BiGS que si recibe el evento de cerrar ventana no finalice la ejecución del programa. La responsable de ello será nuestra aplicación, que deberá consultar si ha ocurrido ese evento con `closeRequested()`.

```
bool closeRequested();
```

Devuelve `true` si se produjo el evento de cerrar ventana y `false` si no. Tras llamar a esta función, y en el caso de que hubiese llegado el evento de cierre, este se resetea de manera que una siguiente llamada a `closeRequested()` devolverá `false` mientras no se reciba un nuevo evento de cierre de ventana.

```
void logEvents(bool f=true);
```

Activa o desactiva mensajes en consola con información sobre los eventos que se van recibiendo.

```
void eventFlush();
```

Vacía la cola de eventos.

```
std::string to_string(const Event &e);
```

Devuelve una cadena con información descriptiva del evento `e`.

7.6.1. Control de teclado y ratón

```
bool eventPresent();
```

Devuelve `true` si hay en cola algún evento relacionado con el teclado o con el ratón y `false` en caso contrario.

```
bool eventNext(Event &ev);
```

Consulta si hay algún evento pendiente. Si lo hay devuelve `true` y el evento en el parámetro `ev`. Si no lo hay devuelve `false`.

```
Event eventWait();
```

Consulta si hay algún evento pendiente. Si lo hay lo devuelve. Si no lo hay espera hasta que lo haya y lo devuelve.

```
Event eventWaitKeyboard();
```

Hace lo mismo que `eventWait()` pero solo espera eventos de teclado. Si mientras tanto se producen eventos de ratón estos se pierden.

```
Event eventWaitMouse();
```

Hace lo mismo que `eventWait()` pero solo espera eventos de ratón. Si mientras tanto se producen eventos de teclado estos se pierden.

```
std::string eventKeyName(int k);
```

Esta función recibe un *keycode* *k* y devuelve una cadena con la descripción de la tecla asociada.

7.6.2. Control del estado del teclado sin eventos

```
bool isKeyPressed(int k);
```

Esta función recibe un *keycode* *k* y devuelve *true* si la correspondiente tecla está pulsada. Devuelve *false* si no está pulsada. El funcionamiento de esta función es independiente del sistema de gestión de eventos.

7.6.3. Códigos de teclas (keycode)

Los códigos de teclas (*keycode*) son los definidos en Allegro 5:

ALLEGRO_KEY_A	ALLEGRO_KEY_END	ALLEGRO_KEY_MENU
...	ALLEGRO_KEY_PGUP	ALLEGRO_KEY_SCROLLLOCK
ALLEGRO_KEY_Z	ALLEGRO_KEY_PGDN	ALLEGRO_KEY_NUMLOCK
ALLEGRO_KEY_0	ALLEGRO_KEY_LEFT	ALLEGRO_KEY_CAPSLOCK
...	ALLEGRO_KEY_RIGHT	ALLEGRO_KEY_PAD_EQUALS
ALLEGRO_KEY_9	ALLEGRO_KEY_UP	ALLEGRO_KEY_BACKQUOTE
ALLEGRO_KEY_PAD_0	ALLEGRO_KEY_DOWN	ALLEGRO_KEY_SEMICOLON2
...	ALLEGRO_KEY_PAD_SLASH	ALLEGRO_KEY_COMMAND
ALLEGRO_KEY_PAD_9	ALLEGRO_KEY_PAD_ASTERISK	
ALLEGRO_KEY_F1	ALLEGRO_KEY_PAD_MINUS	/* Since: 5.1.1 */
...	ALLEGRO_KEY_PAD_PLUS	/* Android only for now */
ALLEGRO_KEY_F12	ALLEGRO_KEY_PAD_DELETE	ALLEGRO_KEY_BACK
ALLEGRO_KEY_ESCAPE	ALLEGRO_KEY_PAD_ENTER	
ALLEGRO_KEY_TILDE	ALLEGRO_KEY_PRINTSCREEN	/* Since: 5.1.2 */
ALLEGRO_KEY_MINUS	ALLEGRO_KEY_PAUSE	/* Android only for now */
ALLEGRO_KEY_EQUALS	ALLEGRO_KEY_ABNT_C1	ALLEGRO_KEY_VOLUME_UP
ALLEGRO_KEY_BACKSPACE	ALLEGRO_KEY_YEN	ALLEGRO_KEY_VOLUME_DOWN
ALLEGRO_KEY_TAB	ALLEGRO_KEY_KANA	
ALLEGRO_KEY_OPENBRACE	ALLEGRO_KEY_CONVERT	/* Since: 5.1.6 */
ALLEGRO_KEY_CLOSEBRACE	ALLEGRO_KEY_NOCONVERT	/* Android only for now */
ALLEGRO_KEY_ENTER	ALLEGRO_KEY_AT	ALLEGRO_KEY_SEARCH
ALLEGRO_KEY_SEMICOLON	ALLEGRO_KEY_CIRCUMFLEX	ALLEGRO_KEY_DPAD_CENTER
ALLEGRO_KEY_QUOTE	ALLEGRO_KEY_COLON2	ALLEGRO_KEY_BUTTON_X
ALLEGRO_KEY_BACKSLASH	ALLEGRO_KEY_KANJI	ALLEGRO_KEY_BUTTON_Y
ALLEGRO_KEY_BACKSLASH2	ALLEGRO_KEY_LSHIFT	ALLEGRO_KEY_DPAD_UP
ALLEGRO_KEY_COMMA	ALLEGRO_KEY_RSHIFT	ALLEGRO_KEY_DPAD_DOWN
ALLEGRO_KEY_FULLSTOP	ALLEGRO_KEY_LCTRL	ALLEGRO_KEY_DPAD_LEFT
ALLEGRO_KEY_SLASH	ALLEGRO_KEY_RCTRL	ALLEGRO_KEY_DPAD_RIGHT
ALLEGRO_KEY_SPACE	ALLEGRO_KEY_ALT	ALLEGRO_KEY_SELECT
ALLEGRO_KEY_INSERT	ALLEGRO_KEY_ALTGR	ALLEGRO_KEY_START
ALLEGRO_KEY_DELETE	ALLEGRO_KEY_LWIN	ALLEGRO_KEY_L1
ALLEGRO_KEY_HOME	ALLEGRO_KEY_RWIN	ALLEGRO_KEY_R1

7.6.4. Constantes para consultar las teclas modificadoras

ALLEGRO_KEYMOD_SHIFT	ALLEGRO_KEYMOD_ALTGR	ALLEGRO_KEYMOD_ACCENT1
ALLEGRO_KEYMOD_CTRL	ALLEGRO_KEYMOD_COMMAND	ALLEGRO_KEYMOD_ACCENT2
ALLEGRO_KEYMOD_ALT	ALLEGRO_KEYMOD_SCROLLLOCK	ALLEGRO_KEYMOD_ACCENT3
ALLEGRO_KEYMOD_LWIN	ALLEGRO_KEYMOD_NUMLOCK	ALLEGRO_KEYMOD_ACCENT4
ALLEGRO_KEYMOD_RWIN	ALLEGRO_KEYMOD_CAPSLOCK	
ALLEGRO_KEYMOD_MENU	ALLEGRO_KEYMOD_INALTSEQ	

Cuando se recibe un evento de tipo *KEY_CHAR* se puede consultar el estado de las teclas modificadoras con una estructura similar a la siguiente:

```
if (e.keyboard.modifiers & ALLEGRO_KEYMOD_SHIFT) cout << "SHIFT" << endl;
```

Se ha de hacer la operación *AND* a nivel de bit (&) con el valor del campo *modifiers* del evento para

averiguar si el bit de la tecla en cuestión está no activo. Tenga cuidado de no confundir el operador & con el operador &&.

7.7. Audio

Aunque Allegro 5 dispone de una gran funcionalidad para la gestión de sonidos, en BiGS lo hemos simplificado mucho. Se ha definido un nuevo tipo:

- **Audio** para representar un sonido cargado desde un fichero.

BiGS establece un máximo de 8 canales de reproducción, es decir, que como mucho admite 8 reproducciones simultáneas de sonidos.

```
Audio audioLoad(const char *f);
```

Carga un sonido desde el fichero `f` y devuelve un dato para poder utilizarlo con `audioPlay()` o `audioStop()`.

```
void audioPlay(Audio a, int id, float vol=1, float bal=0, float vel=1, PlayMode m=PLAY_ONCE);
```

Reproduce el sonido `a` en el canal `id`, que será un valor entre 0 y 7.

`vol` es la ganancia (volumen) de reproducción. 1 es un volumen normal.

`bal` es el balance. Es un valor real entre -1 y 1 siendo -1 izquierda, 0 centro y 1 derecha.

`vel` es la velocidad de reproducción. 1 es la velocidad normal.

`m` es el modo de reproducción y puede valer alguno de estos valores:

- **PLAY_ONCE** para reproducir el sonido una única vez.
- **PLAY_LOOP** para reproducir el sonido indefinidamente en bucle. Esto puede utilizarse, por ejemplo, para reproducir alguna música de fondo en un videojuego.
- **PLAY_BIDIR** para reproducir el sonido primero normal y luego invertido.

```
void audioStop(int id=-1);
```

Detiene la reproducción en el canal indicado (valor entre 0 y 7). Si en canal no se pasa nada (o bien -1) se detienen todos los canales.

7.8. Control del reloj

```
void timeDelay(long int ms);
```

Detiene la ejecución del programa durante `ms` milisegundos.

```
double timeGet();
```

BiGS inicia un cronómetro al comienzo de la ejecución de nuestro programa. Con esta función podemos saber el tiempo transcurrido (en microsegundos) desde ese momento de inicio.

```
double timeFromLastCall();
```

Esta función devuelve del tiempo transcurrido (en microsegundos) desde la última vez que fue llamada. La primera vez que se llama devuelve cero.

8. Compilación y enlazado en línea de órdenes

En sistemas GNU/Linux puede hacer la compilación y el enlazado usando BiGS con las opciones que se indican a continuación. Como ejemplo usaremos el programa `circulos.cpp` proporcionado en los ficheros de prueba.

Compilación

```
g++ -c circulos.cpp -o circulos.o -I../bigs/include
```

Enlazado

```
g++ circulos.o -lbigs -lallegro_font -lallegro_ttf -lallegro_audio -lallegro_image  
-lallegro_primitives -lallegro_acodec -lallegro -lallegro_main  
-L../bigs/lib  
-o circulos
```